Bidirectional RNNs

Contents

1. Overview . . . 2
2. Prerequisites . . . 2
3. Why read a sequence in both directions . . . 3
   - 3.1 The limit of a unidirectional RNN . . . 3
   - 3.2 An example: named entity recognition . . . 3
4. How a bidirectional RNN works . . . 4
   - 4.1 Two RNNs, opposite directions . . . 4
   - 4.2 Why the output now sees the future . . . 5
   - 4.3 The equations . . . 6
5. Bidirectional RNNs in Keras . . . 7
   - 5.1 The wrapper . . . 7
   - 5.2 Counting the parameters . . . 8
6. Experiment: tagging the parts of speech . . . 9
   - 6.1 The task and the models . . . 9
   - 6.2 Results . . . 9
   - 6.3 Where the gain comes from . . . 10
7. Applications . . . 13
8. Disadvantages . . . 13
9. Summary . . . 15
10. Sources . . . 15
11. Key terms . . . 16

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Recurrent neural network (RNN); LSTM (long short-term memory); GRU (gated recurrent unit).

1. Overview

Key point: A bidirectional RNN runs two RNNs over the same sequence, one from left to right and one from right to left, and joins their hidden states at every time step. Each output can then use both the past and the future of the sequence.

An RNN keeps a **hidden state** (G-891), its summary of the inputs read so far, and updates it at every **time step** (G-1976), one word at a time. A **bidirectional RNN** (G-292) (Schuster and Paliwal 1997) combines an RNN that moves forward through time, from the start of the sequence, with another that moves backward through time, from the end (Goodfellow §10.3). The RNNs of the earlier Notes read only from left to right, so the output at a time step can depend only on the inputs up to that step. A bidirectional RNN removes that limit.

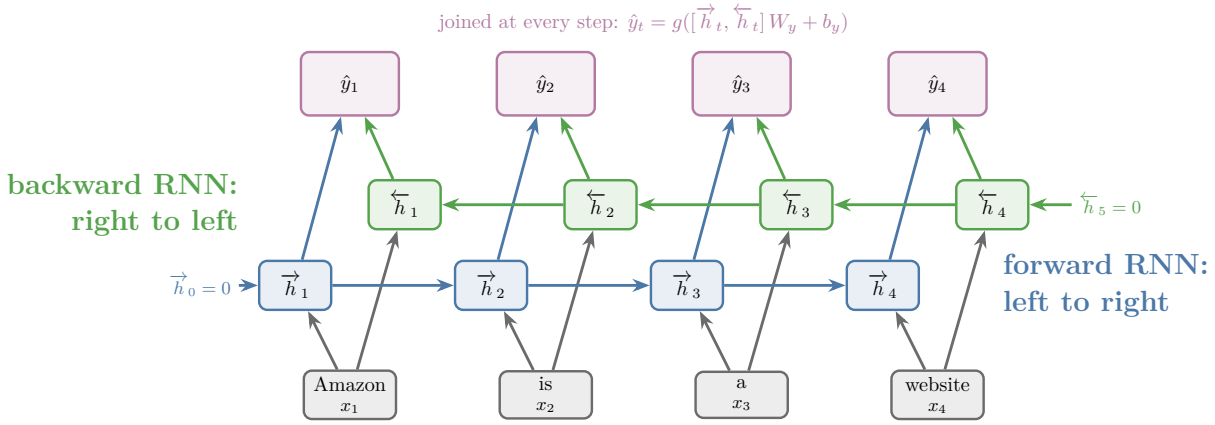


Figure 1: A bidirectional RNN on "Amazon is a website". The forward RNN (blue) reads left to right; the backward RNN (green) reads right to left. At every time step their two hidden states are joined to give the output, so $\hat{y}_1$, the output for "Amazon", already depends on "website"

Figure 1 shows the structure. The idea works with any recurrent layer: with LSTM layers it is called a **BiLSTM** (G-295), with GRU layers a **BiGRU**.

2. Prerequisites

- The RNN formulas and counting the parameters: $h_t = \tanh(x_t W_i + h_{t-1} W_h + b_h)$ and

counting parameters.
- The IMDB dataset and the `Embedding` layer.
- Many-to-many models: **many-to-many** (G-1154) models and `return_sequences=True`; part-of-speech tagging and **named entity recognition** (G-1300).
- The LSTM cell and the four steps of the GRU: the gated layers that can be made bidirectional.
- The architecture of a deep RNN: stacking recurrent layers.

3. Why read a sequence in both directions

> **Key point:** In a **unidirectional RNN** (G-2042) the output at time t depends only on the inputs up to t. Some tasks need later inputs to decide an earlier output.

3.1 The limit of a unidirectional RNN

> **Key point:** Information flows only forward, so the past can affect an output but the future cannot.

Every RNN so far is **unidirectional** (G-2042): it reads x_1, then x_2, then x_3. The output at the last step depends on x_3, x_2 and x_1: on all the past inputs. Goodfellow §10.3 calls this a "causal" structure: the state at time t captures only information from the past and the present input.

Now suppose the correct output at an early **time step** (G-1976) depends on an input that comes later. A unidirectional RNN, whether simple, LSTM or GRU, has not yet read that input when it must give the output, so it fails.

3.2 An example: named entity recognition

> **Key point:** In "I love Amazon. It's a great website" the word Amazon is an organisation; in "I love Amazon. It's a beautiful river" it is a location. Reading left to right, the words that decide come after Amazon.

Named entity recognition (G-1300) (NER) is the task of labelling the names in a sentence with their type: a person, a location, an organisation (see many-to-many models). Chatbots use it to pick names and places out of messages. Take two sentences:

- "I love Amazon. It's a great website." Here Amazon is an **organisation**.
- "I love Amazon. It's a beautiful river." Here Amazon is a **location**.

A tagger reading left to right reaches Amazon with exactly the same context, "I love", in both sentences. Until it reads the next sentence, it cannot tell the organisation from the river.

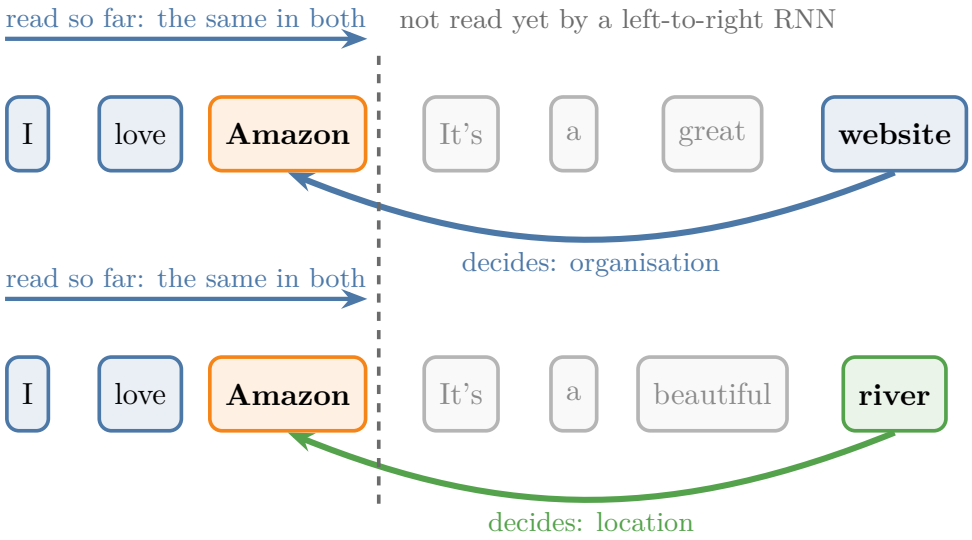


Figure 2: The two sentences as a left-to-right RNN meets them. Up to "Amazon" both read "I love Amazon"; the word that decides the label, "website" or "river", comes later, on the far side of the dashed line

In Figure 2, everything left of the dashed line is identical in the two sentences, so a left-to-right tagger must give "Amazon" the same label in both. The next inputs decide the output at an earlier step: that is the situation a **bidirectional RNN** (G-292) is built for. **Machine translation** (G-1141) has the same need: a word of the output may depend on parts of the input that come later.

Goodfellow §10.3 gives a speech example: the correct interpretation of the current sound may depend on the next few sounds, and even on the next few words.

4. How a bidirectional RNN works

> **Key point:** Two separate RNNs read the sequence in opposite directions. At every **time step** (G-1976) their hidden states are concatenated, and the output layer reads the joined vector.

4.1 Two RNNs, opposite directions

> **Key point:** The forward RNN starts at x_1 from a vector of zeros; the backward RNN starts at x_T (the last of the T inputs) from a vector of zeros.

Take the four words "Amazon is a website" as x_1, x_2, x_3, x_4.

1. **The forward RNN** (blue in Figure 1) is the RNN we know. The forward RNN starts from $\overrightarrow{h}_0$, a vector of zeros, reads Amazon, is, a, website, and produces $\overrightarrow{h}_1, \ldots, \overrightarrow{h}_4$.

2. **The backward RNN** (green) is a second, separate RNN with its own weights. The backward RNN starts from zeros at the other end, reads website, a, is, Amazon, and produces $\overleftarrow{h}_4, \ldots, \overleftarrow{h}_1$.
3. **At every time step**, the two **hidden states** (G-891) are **concatenated** (G-436; placed one after the other in a single vector), and the output layer turns the joined vector into $\hat{y}_t$.

The two RNNs do not feed each other: the outputs of the forward states are not connected to the inputs of the backward states, and the other way round (Schuster and Paliwal 1997). They meet only at the output.

4.2 Why the output now sees the future

> **Key point:** $\overleftarrow{h}_1$ is the backward RNN's last state: it has read every word. So $\hat{y}_1$ depends on the whole sentence.

Look at the output for Amazon, $\hat{y}_1$, in Figure 1.

- Its forward part, $\overrightarrow{h}_1$, has read only Amazon.
- Its backward part, $\overleftarrow{h}_1$, has read website, a, is and Amazon: it is the last state of the backward RNN.

So every word, including "website", contributes to the label of Amazon. "Website" points to an organisation, not a location. In general, the output at each step gets a summary of the past from the forward RNN and a summary of the future from the backward RNN, while staying most sensitive to the inputs around time t (Goodfellow §10.3).

Figure 3 shows the two passes on a real test sentence from the experiment of section 6. Watch the word "about": the forward pass reaches it having read only "…that totaled about", and the **unidirectional** (G-2042) LSTM tags it as a preposition (IN). The backward pass reaches "about" having already read the "$" that follows, and the BiLSTM tags it correctly as an adverb (RB).

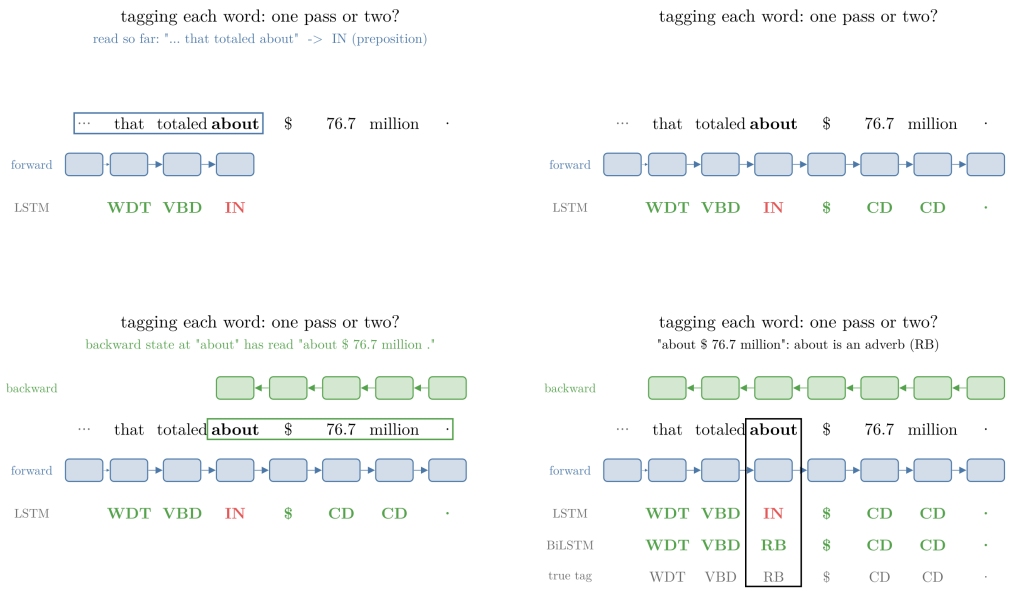


Figure 3: Tagging a real CoNLL-2000 test sentence. The forward pass (blue) sweeps left to right and the unidirectional LSTM gives its tags (red: wrong). The backward pass (green) then sweeps right to left from the last word. Joining both states, the BiLSTM tags "about" RB, the true tag, because it has read "$ 76.7 million". Tags from the Notebook's LSTM (64 nodes) and BiLSTM, seed 1

4.3 The equations

> **Key point:** The forward equation uses $\overrightarrow{h}_{t-1}$, the backward equation uses $\overleftarrow{h}_{t+1}$, and the output uses both, joined.

1. **In words:** the forward state at t comes from the input and the forward state at $t-1$. The backward state at t comes from the input and the backward state at $t+1$, the step it read just before. The output applies its activation to the joined pair.
2. **Formula:** the arrow over a symbol shows the direction its RNN reads: $\rightarrow$ left to right, $\leftarrow$ right to left. Each RNN has its own weights and bias:

$$\overrightarrow{h}_t = \tanh\left(x_t \overrightarrow{W}_i + \overrightarrow{h}_{t-1} \overrightarrow{W}_h + \overrightarrow{b}\right)$$

$$\overrightarrow{h}_0 = 0$$

$$\overleftarrow{h}_t = \tanh\left(x_t \overleftarrow{W}_i + \overleftarrow{h}_{t+1} \overleftarrow{W}_h + \overleftarrow{b}\right)$$

$$\overleftarrow{h}_{T+1} = 0$$

$$\hat{y}_t = g\left(\left[\overrightarrow{h}_t, \overleftarrow{h}_t\right] W_y + b_y\right)$$

where $[\,\cdot\,,\,\cdot\,]$ joins two row vectors into one and g is the output activation, a sigmoid or a softmax.

3. **Example:** with 2 nodes in each direction, suppose $\overrightarrow{h}_1 = [0.3, -0.1]$ and $\overleftarrow{h}_1 = [0.6, 0.2]$. The output layer receives the 4 numbers

$$[\overrightarrow{h}_1, \overleftarrow{h}_1] = [0.3, -0.1, 0.6, 0.2]$$

so W_y has 4 rows: twice as many as for one direction. With $W_y = [1.0, -1.0, 0.5, 2.0]^\top$ (the $\top$ stands the row up as a column of 4 weights), $b_y = 0$ and the sigmoid σ for g, one product per line:

$$0.3 \times 1.0 = 0.30$$

$$-0.1 \times (-1.0) = 0.10$$

$$0.6 \times 0.5 = 0.30$$

$$0.2 \times 2.0 = 0.40$$

$$0.30 + 0.10 + 0.30 + 0.40 = 1.10$$

$$\hat{y}_1 = \sigma(1.10) = 0.75$$

The Notebook builds a Keras bidirectional layer, runs the forward and backward equations by hand with its weights, and gets exactly its output at every **time step** (G-1976).

5. Bidirectional RNNs in Keras

> **Key point:** Wrap any recurrent layer in `keras.layers.Bidirectional`. The wrapper makes a second copy for the backward direction, so the layer's parameters double.

5.1 The wrapper

> **Key point:** One line changes: `SimpleRNN(5)` becomes `Bidirectional(SimpleRNN(5))`.

> **Python:** The IMDB model with a `SimpleRNN`, made bidirectional.

```
model = keras.Sequential([
    keras.Input(shape=(100,)),
    keras.layers.Embedding(10000, 32),
    keras.layers.Bidirectional(keras.layers.SimpleRNN(5)),
    keras.layers.Dense(1, activation="sigmoid")])
```

By default the wrapper concatenates the two directions (`merge_mode="concat"`); it can also add, average or multiply them (Keras documentation, `Bidirectional`). Without `return_sequences=True`, as here, it returns the forward RNN's last state joined with the backward RNN's last state, which is $\overleftarrow{h}_1$.

5.2 Counting the parameters

> **Key point:** Two RNNs instead of one: 190 becomes 380 for `SimpleRNN(5)`. The layer after it also gets twice as many inputs.

Recurrent layer (5 nodes, input 32)	Unidirectional	Bidirectional
`SimpleRNN`	190	380
`LSTM`	760	1,520
`GRU`	585	1,170

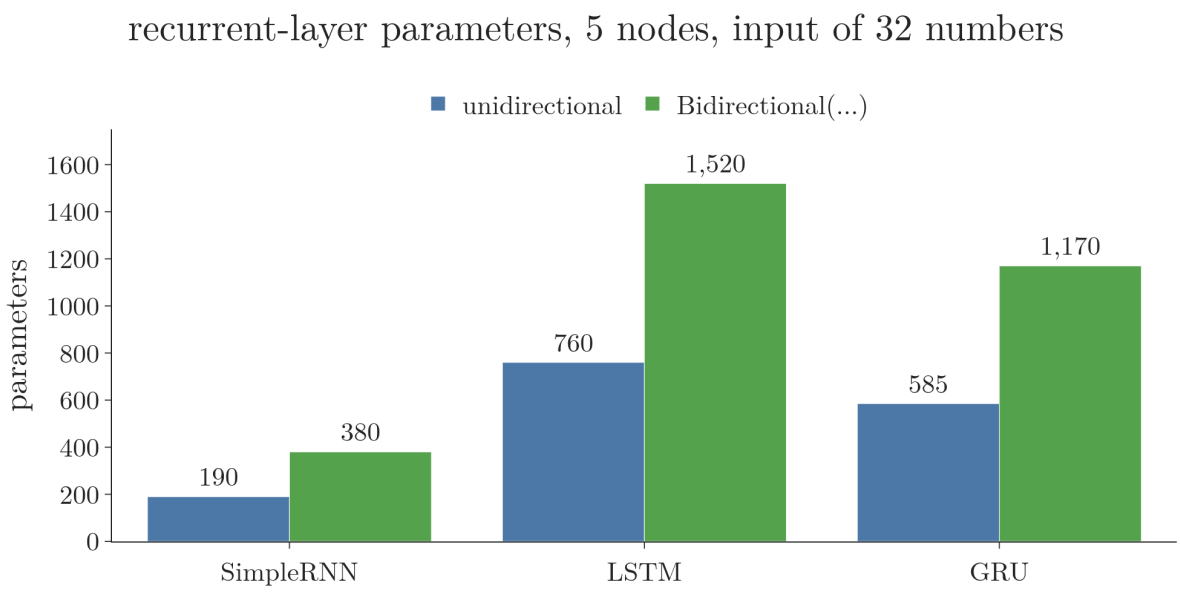


Figure 4: Parameters of a recurrent layer with 5 nodes on a 32-number input, before and after wrapping it in `Bidirectional`. Every bar doubles

Figure 4 shows the doubling for all three layer types: the wrapper adds a second, separate copy of the layer for the backward direction.

The `Dense(1)` output layer grows too, because it reads 10 joined numbers instead of 5:

$$5 + 1 = 6 \quad \text{becomes} \quad 10 + 1 = 11$$

That is 6 parameters before, 11 after.

Wrapping an `LSTM` gives a **BiLSTM** (G-295) and wrapping a `GRU` gives a BiGRU. The bidirectional simple RNN is rarely used; BiLSTMs and BiGRUs are the common choices in practice, for the reasons given in the vanishing gradient through time.

6. Experiment: tagging the parts of speech

> **Key point:** On real Wall Street Journal sentences, a BiLSTM tags parts of speech more accurately than a **unidirectional** (G-2042) LSTM, also when the unidirectional one gets the same number of parameters. Most of the gain is on words whose tag depends on what follows.

6.1 The task and the models

> **Key point:** 8,936 training sentences, 2,012 test sentences, 44 tags. Every model reads a sentence and gives one tag per word: a **many-to-many** (G-1154) task.

Part-of-speech tagging (G-1456) labels every word of a sentence with its grammatical class: noun (NN), plural noun (NNS), verb in the past tense (VBD), preposition (IN), adverb (RB) and so on. The data is the CoNLL-2000 corpus (a collection of labelled text): Wall Street Journal sentences, each word labelled with one of 44 tags (Tjong Kim Sang and Buchholz 2000). A copy of the word and tag columns is stored in the Note's `data` folder (from the NLTK data collection), so the Notebook runs without a download.

- Each sentence is padded to 80 words (the longest has 78). Words seen fewer than 2 times in training share one "unknown" id, giving a vocabulary of 9,676.
- Every model is `Embedding(9676, 64)`, a recurrent layer with `return_sequences=True`, and a `Dense(44, activation="softmax")` layer that gives 44 probabilities (one per tag) at every **time step** (G-1976) (see many-to-many models).
- Padding positions get a sample weight of 0, so they count neither in the loss nor in the accuracy.
- Adam (a version of gradient descent), 8 epochs (8 passes over the training sentences), batch size 64 (64 sentences per weight update), 3 seeds per model (3 runs from different random starting weights).

Model	Recurrent parameters
LSTM, 64 nodes	33,024
BiLSTM, 64 + 64 nodes	66,048
LSTM, 100 nodes	66,000

The third model checks that the gain is not only from the doubled parameters: it is a **unidirectional** (G-2042) LSTM with almost the same number of parameters as the BiLSTM.

6.2 Results

> **Key point:** BiLSTM 94.7% against 93.0% (LSTM, 64 nodes) and 93.2% (LSTM, 100 nodes). Doubling the parameters of a **unidirectional** (G-2042) LSTM gains only 0.2 points; at the same parameter count, reading backwards gains 1.5.

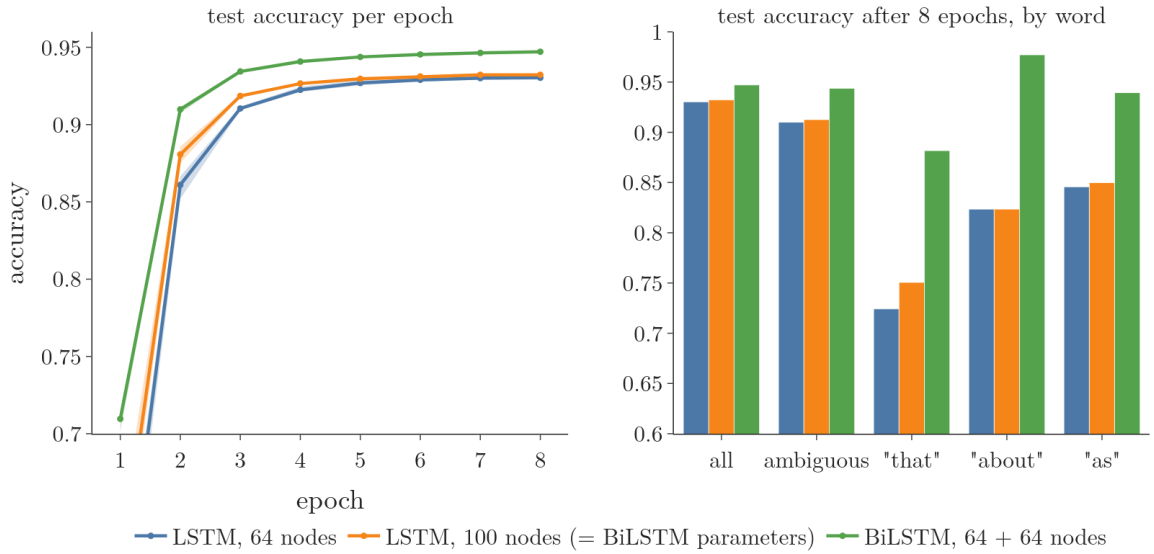


Figure 5: Left: test accuracy per epoch, mean of 3 seeds (band: lowest to highest seed). Right: test accuracy after 8 epochs on all words, on the ambiguous words, and on three ambiguous words separately

Model	Test accuracy, mean of 3 seeds	Lowest	Highest
LSTM, 64 nodes	0.930	0.929	0.932
LSTM, 100 nodes (same parameters as BiLSTM)	0.932	0.932	0.933
BiLSTM, 64 + 64 nodes	**0.947**	0.947	0.947

- The **BiLSTM** (G-295) is ahead at every epoch (Figure 5, left), and the three seeds of each model barely differ.
- The wider unidirectional LSTM gains only 0.2 points over the narrow one:

$$0.932 - 0.930 = 0.002$$

- The BiLSTM has the same number of parameters as the wider LSTM and gains 1.5 points over it:

$$0.947 - 0.932 = 0.015$$

- Only the reading direction differs between those two models, so the gain comes from the backward direction, not from the extra parameters.

6.3 Where the gain comes from

> **Key point:** On “that”, “about” and “as”, whose tag depends on the next words, the **BiLSTM** (G-295) gains 9 to 16 points.

Some words take different tags in different sentences. Call a word **ambiguous** if, in the training sentences, its second most common tag covers at least 10% of its uses. These words make up 17.3% of the test words.

Words	LSTM, 64	LSTM, 100	BiLSTM
all words	0.930	0.932	0.947
ambiguous words	0.910	0.913	0.944
"that" (417 in test)	0.724	0.751	0.882
"about" (102)	0.824	0.824	0.977
"as" (242)	0.846	0.850	0.939

In the training sentences, the tag of these words goes with the tag of the **next** word, which a **unidirectional** (G-2042) model has not read yet (counts from the Notebook's training data):

Word and tag	Uses	Most common next tags
"that" as a relative pronoun (WDT): "the deal that collapsed"	476	verbs: VBD 141, VBZ 120, VBP 102, MD 101
"that" as a determiner (DT): "that deal"	272	NN 166
"that" as a conjunction (IN): "said that the deal"	1,053	DT 320, PRP 175
"about" as an adverb (RB)	70	"$" in all 70: "about $ 5 million"
"about" as a preposition (IN)	371	CD 140, DT 79
"as" as an adverb (RB): "as much as"	117	JJ 67, RB 47
"as" as a preposition (IN)	789	DT 233, NN 96

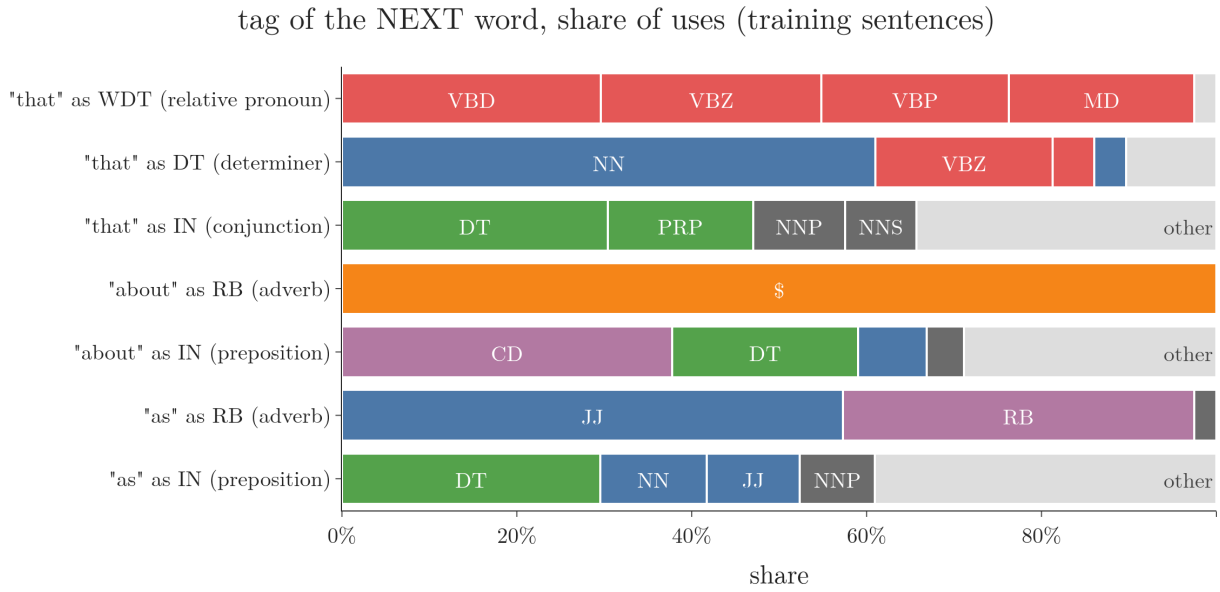


Figure 6: For each tag of "that", "about" and "as", the tag of the next word in the training sentences, as a share of uses. Red marks verbs. Each tag of a word has its own typical followers

Figure 6 draws the table. Compare the rows of one word: "that" as WDT is followed by a verb, "that" as DT mostly by a noun (NN), and "about" as RB always by "$". The next tag separates the uses of the word.

The previous word is a weaker clue: before "that", a noun is common for both WDT (NN 228) and IN (NN 195). The words after decide, and only the backward RNN has read them.

One test sentence, tagged by the LSTM with 64 nodes and by the BiLSTM (seed 1):

Word	True tag	LSTM	BiLSTM
In	IN	IN	IN
the	DT	DT	DT
$	$	$	$
300-a-share	JJ	JJ	JJ
buyout	NN	NNS	NN
,	,	,	,
that	WDT	WDT	WDT
totaled	VBD	VBD	VBD
about	**RB**	**IN**	**RB**
$	$	$	$
76.7	CD	CD	CD
million	CD	CD	CD
.	.	.	.

The unidirectional LSTM tags "about" as a preposition: it has read only "...that totaled about". The BiLSTM has also read the "$" that follows, and in the training sentences every adverb "about" comes before "$".

7. Applications

> **Key point:** Use a bidirectional RNN when the whole sequence is available and an output may depend on later inputs: tagging, **named entity recognition** (G-1300), translation, and often sentiment analysis.

1. **Named entity recognition**, as in section 3.2.
2. **Part-of-speech tagging**, as in section 6: each word gets its grammatical class.
3. **Machine translation.** Google's neural **machine translation** (G-1141) system made the first layer of its encoder bidirectional, to have the best possible context at each point (Wu et al. 2016).
4. **Sentiment analysis. Bidirectional RNNs** (G-292) are worth trying here too; the whole review is available before the prediction.

Bidirectional RNNs have been extremely successful in handwriting recognition, speech recognition and bioinformatics (Goodfellow §10.3).

8. Disadvantages

> **Key point:** Twice the parameters, so more training time and more overfitting risk; and the whole sequence must be available before any output, which adds latency in real-time tasks.

1. **Complexity.** The recurrent layer has twice the parameters, so training takes longer and the network can fit the training data too closely (**overfitting**, G-1429). The usual remedies apply: **dropout** (G-639) and regularisation.
2. **The whole sequence must be available.** The backward RNN starts at the last input. In **real-time speech recognition**, the words arrive one by one while the person speaks, so the backward RNN cannot start until the sentence is finished. The reply is delayed: a **latency** (G-1048) problem that a **unidirectional RNN** (G-2042) does not have. Figure 7 shows the delay on a four-word command. Watch the two rows as the words arrive: the unidirectional RNN gives each output when its word arrives, while the bidirectional RNN keeps waiting and gives all four outputs only after the last word.

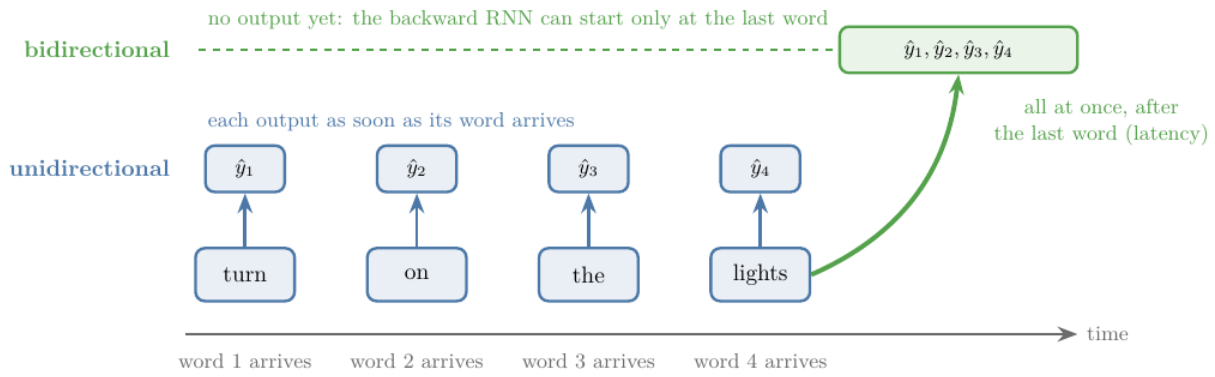


Figure 7: The same four words arriving one by one. Top: a bidirectional RNN can give no output until the last word, because its backward pass starts there. Bottom: a unidirectional RNN gives each output as its word arrives

The same constraint limits parallel computation: Google's translation system kept only its bottom encoder layer bidirectional, because a layer above a bidirectional one must wait for both directions to finish (Wu et al. 2016).

9. Summary

	Unidirectional RNN	Bidirectional RNN
Reads	left to right	left to right and right to left
Output at time t depends on	$x_1, \ldots, x_t$	the whole sequence $x_1, \ldots, x_T$
Recurrent layers	1	2 separate ones, joined at every step
Parameters	P (those of one RNN)	$2P$ (and a wider next layer)
Keras	`LSTM(64)`	`Bidirectional(LSTM(64))`
Needs the full sequence first	no	yes: latency in real-time tasks

- A bidirectional RNN joins a forward RNN and a backward RNN at every **time step** (G-1976): $\hat{y}_t = g([\overrightarrow{h}_t, \overleftarrow{h}_t]W_y + b_y)$, so each output gets a summary of the past and a summary of the future.
- The backward state at t comes from $\overleftarrow{h}_{t+1}$, so every output sees the future of the sequence.
- It helps when an output depends on later inputs: tagging, NER, translation, because a left-to-right RNN has not read those inputs yet when it must give the output; on **part-of-speech tagging** (G-1456) the BiLSTM beat an LSTM with the same number of parameters by 1.5 points.
- It doubles the parameters and needs the whole sequence before any output, because it holds two separate RNNs and the backward one starts at the last input; this adds **latency** (G-1048) in real-time tasks.

10. Sources

Built from

- CampusX, "Bidirectional RNN | BiLSTM | Bidirectional LSTM | Bidirectional GRU", YouTube, https://www.youtube.com/watch?v=k2NSm3MNdYg

Other references

- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press. Chapter 10, §10.3 (bidirectional RNNs). deeplearningbook.org/contents/rnn.html.
- Schuster, M. and Paliwal, K. K. (1997). Bidirectional Recurrent Neural Networks. *IEEE Transactions on Signal Processing* 45(11), 2673–2681.
- Tjong Kim Sang, E. F. and Buchholz, S. (2000). Introduction to the CoNLL-2000 Shared Task: Chunking. CoNLL 2000. (The tagged Wall Street Journal sentences.)
- Wu, Y., Schuster, M., Chen, Z., Le, Q. V., Norouzi, M. et al. (2016). Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation. arXiv:1609.08144.
- Keras API documentation: `Bidirectional` layer (`merge_mode`, default `"concat"`), keras.io/api/layers/recurrent_layers/bidirectional.

11. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Bidirectional RNN (G-292)	Two RNNs reading a sequence in opposite directions, their hidden states joined at every time step, so the output at each word uses the words both before and after it.
BiLSTM, BiGRU (G-295)	A bidirectional RNN made of LSTM or GRU layers.
Unidirectional RNN (G-2042)	An RNN that reads in one direction only, so its output at t depends only on inputs up to t.
`Bidirectional` (G-67)	The Keras wrapper that makes any recurrent layer read the sequence both forwards and backwards (bidirectional).
Bidirectional encoder (G-291)	An encoder with one RNN reading forwards and one backwards, whose states are joined at each position.
Forward and backward RNN (G-796)	The left-to-right and the right-to-left halves of a bidirectional RNN.
Hidden state (h_t) (G-891)	The RNN's running summary of the inputs so far: the recurrent layer's output at time step t.
Time step (G-1976)	One position in the sequence; word j enters at $t = j$.
Many-to-many RNN (G-1154)	An RNN that takes a sequence and produces a sequence; also called sequence-to-sequence.
Named entity recognition (NER) (G-1300)	Marking which words of a sentence are entities, specific things a program must act on, such as "7 pm" and "airport"; chatbots use it to pick out times and places.
Machine translation (G-1141)	Translating a sentence from one language into another.
Concatenation ($[h_{t-1}, x_t]$) (G-436)	Joining two vectors end to end into one longer vector, so one layer can read both at once: 3 numbers and 4 numbers give 7.
Part-of-speech tagging (G-1456)	Labelling every word of a sentence with its part of speech.
Overfitting (G-1429)	Learning the training data too closely, noise included; fails on new data.

Term	Meaning
Dropout (G-639)	Switching off a random set of input and hidden nodes at every training step, to reduce overfitting.
Latency (G-1048)	The delay between a request and its answer; high for KNN on large data.
Observation (G-1374)	One record of a dataset: one row of the data table.
Target (G-1949)	The output column we predict, such as the class; also called the label.